Multimedia Application

By

Minhaz Uddin Ahmed, PhD

Department of Computer Engineering

Inha University Tashkent.

Email: minhaz.ahmed@gmail.com

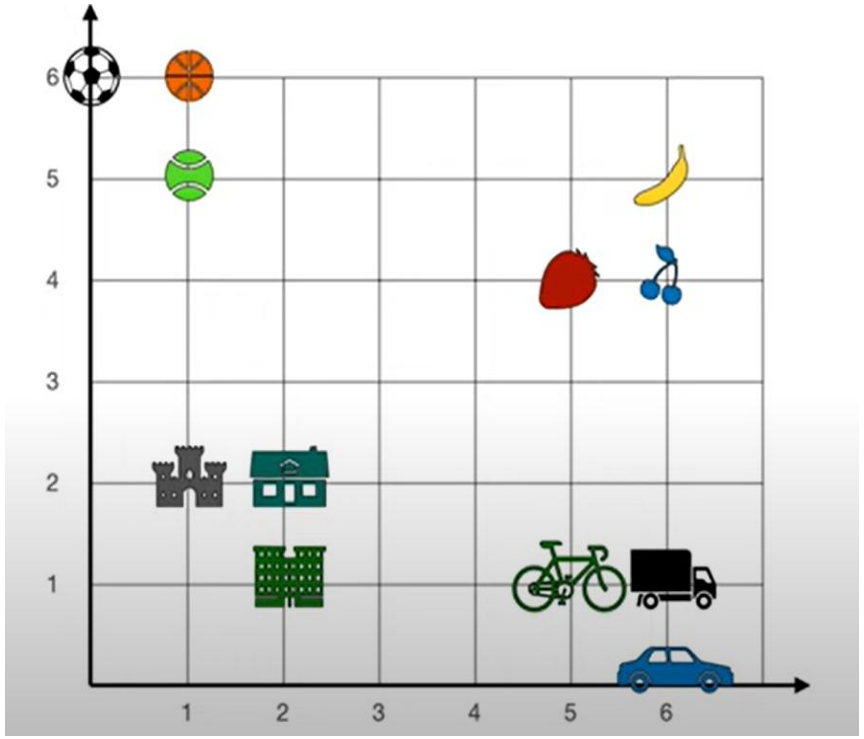
Content

- The Transformer: A Self-Attention Network
- Multi-head Attention
- Transformer Blocks
- The Residual Stream view of the Transformer Block
- The input: embeddings for token and position
- The Language Modeling Head
- Large Language Models with Transformers
- Large Language Models: Generation by Sampling
- Large Language Models: Training Transformers .

Embeddings

Word embedding as a dense representation of words in the form of vectors

Embeddings

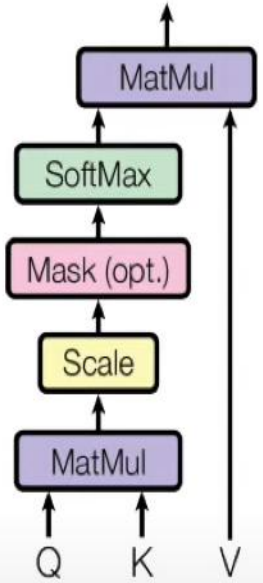


Group of words

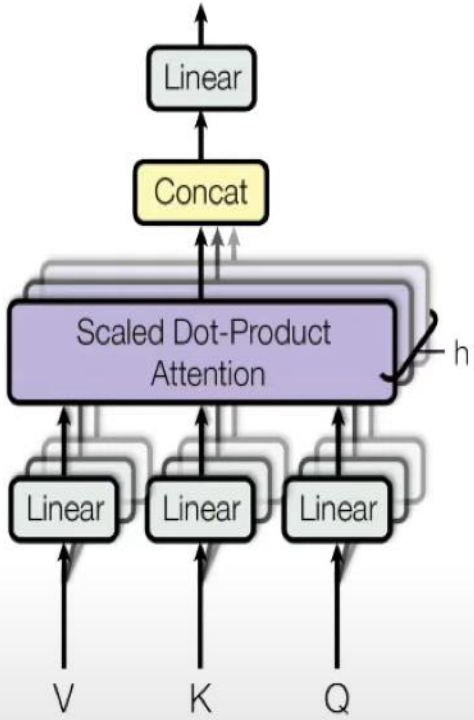
Word	Numbers	
Apple	?	?
Banana	6	5
Strawberry	5	4
Cherry	6	4
Soccer	0	6
Basketball	1	6
Tennis	1	5
Castle	1	2
House	2	2
Building	2	1

Attention

Scaled Dot-Product Attention

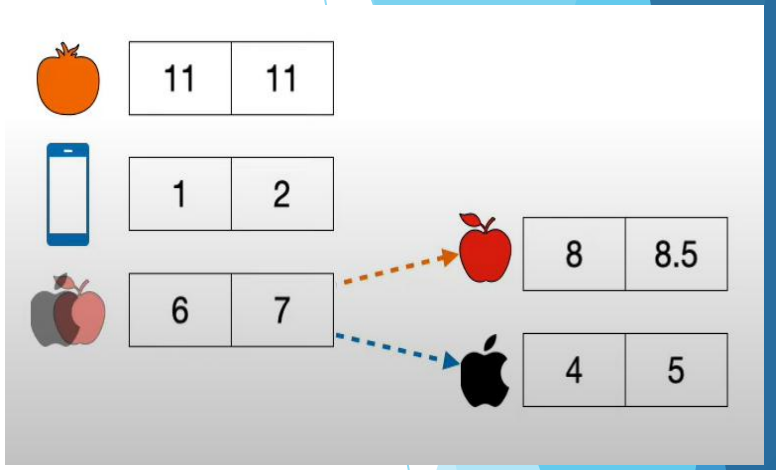


Multi-Head Attention



$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O$$
$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



Please buy an apple and an orange.

Apple unveiled the new phone.

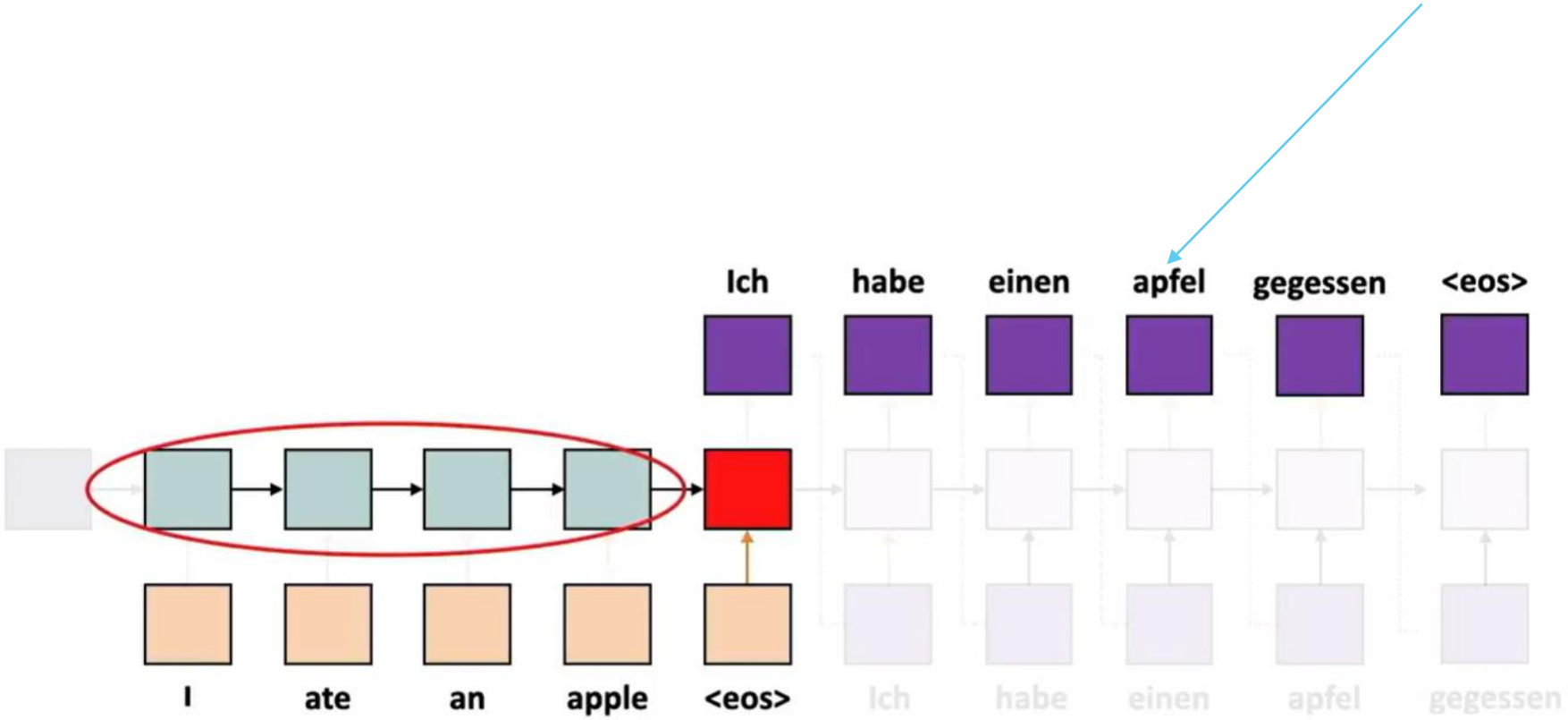
Attention (Information overload)

...to reach the official residence of the Chancellor of the Exchequer in time for the press conference held for the reporters

...rechtzeitig zur Pressekonferenz für die Reporter den Amtssitz des Schatzkanzlers zu erreichen

Will the hidden state remember the beginning of the original sentence in order to provide the right translation at the end of the output sentence?

Single hidden state

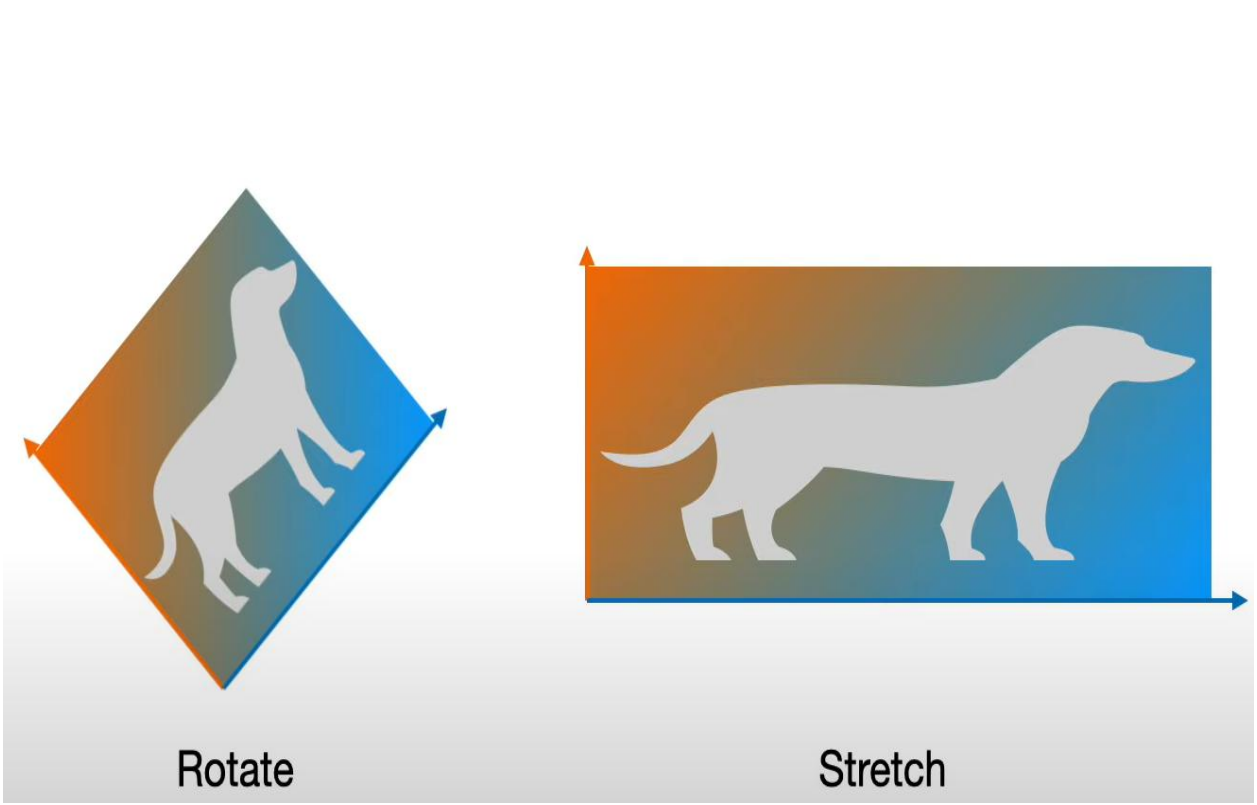


Different hidden values are related to different parts of the output

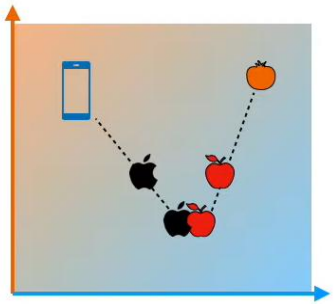
Attention

Attention models help pay attention to relevant portions of the input at the output.

Linear transformation

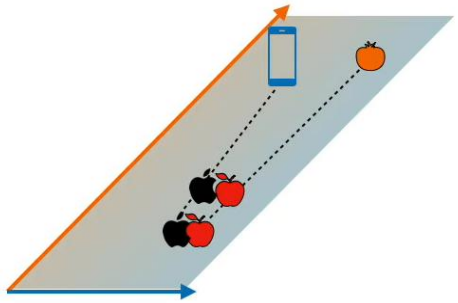


Get new embedding from existing ones.



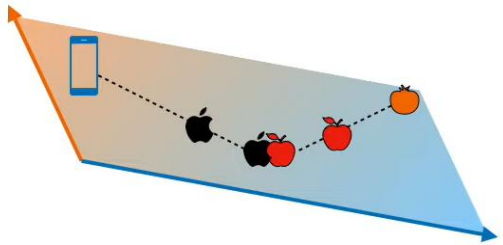
Okay

Score: 1



Bad

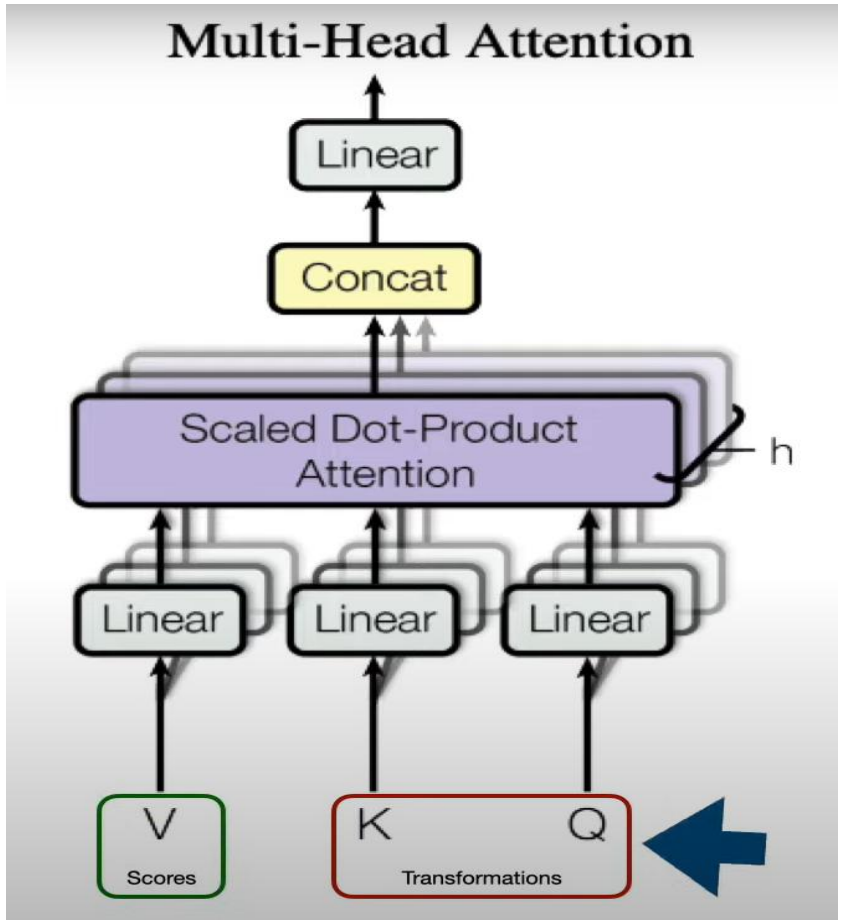
Score: 0.1



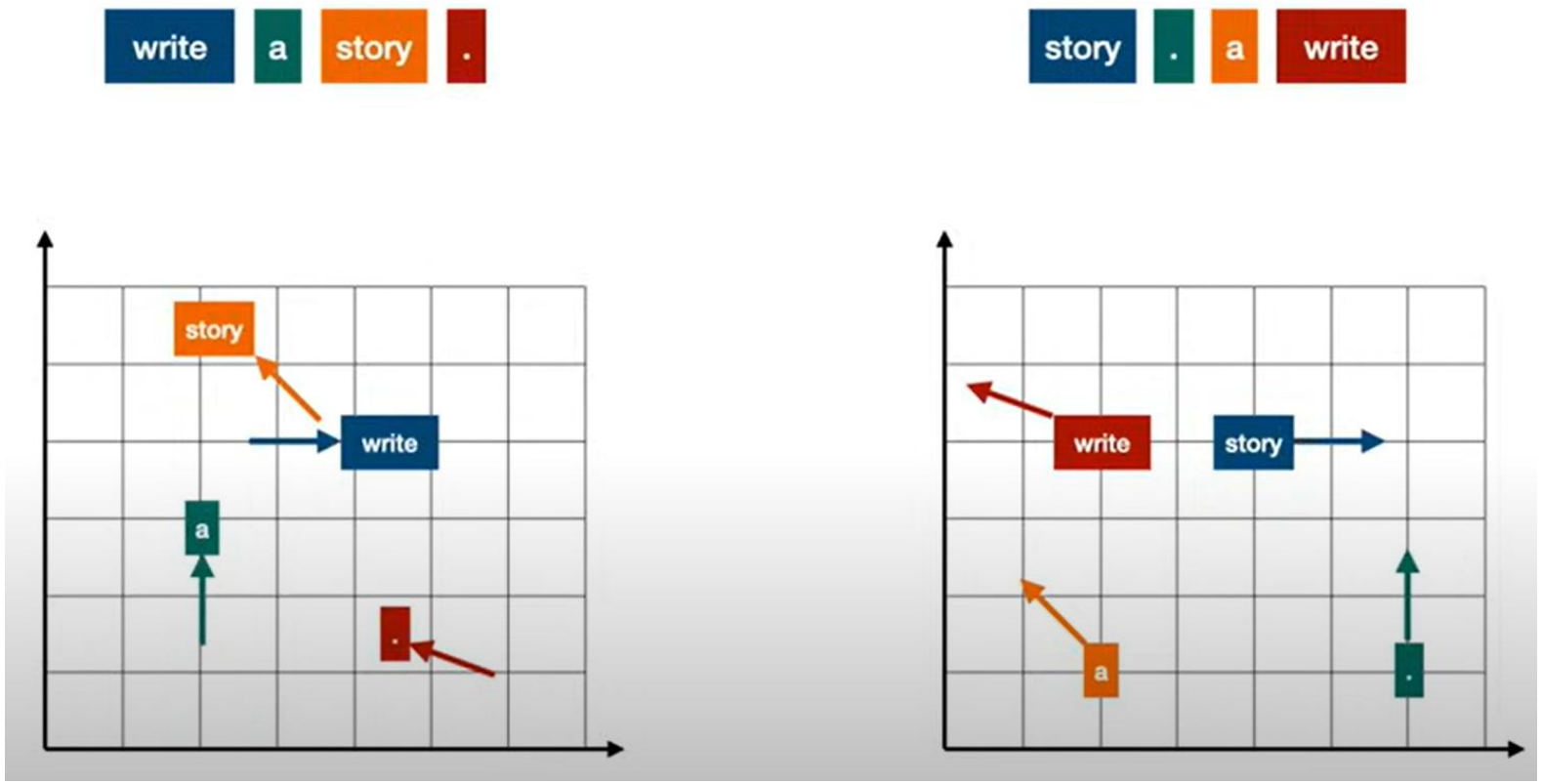
Good

Score: 4

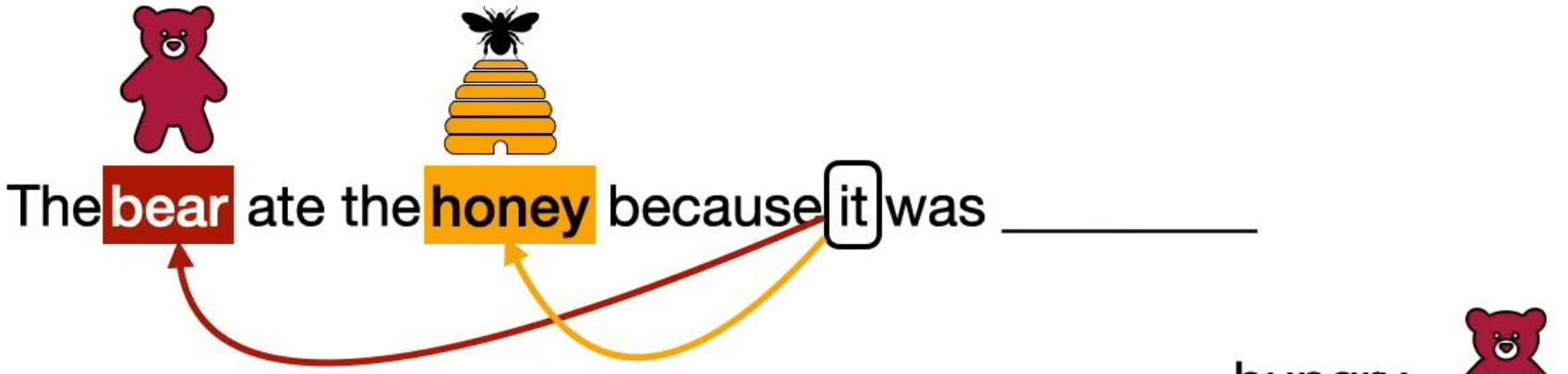
Multi-Head Attention



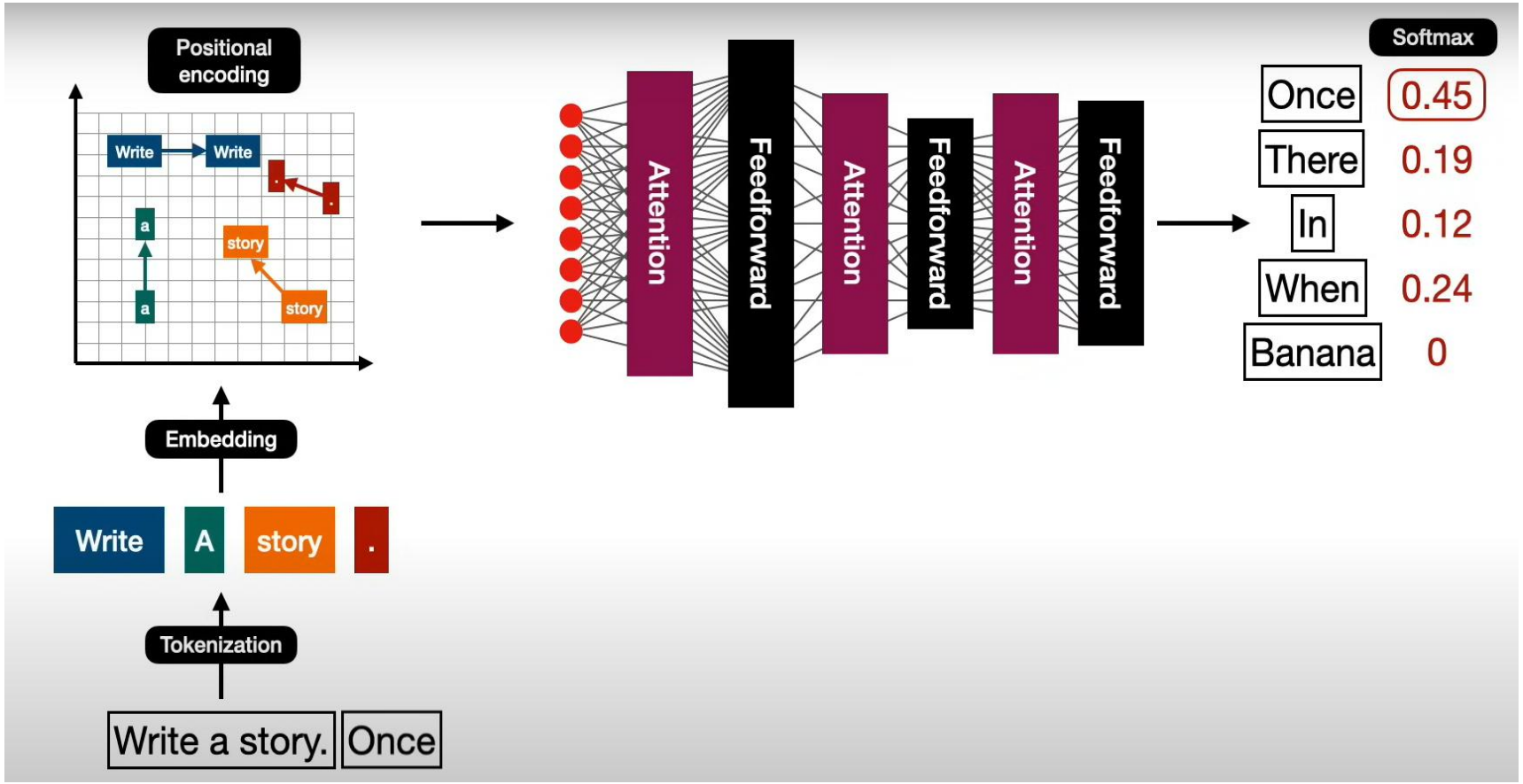
Positional encoding



Using context



Transformer model



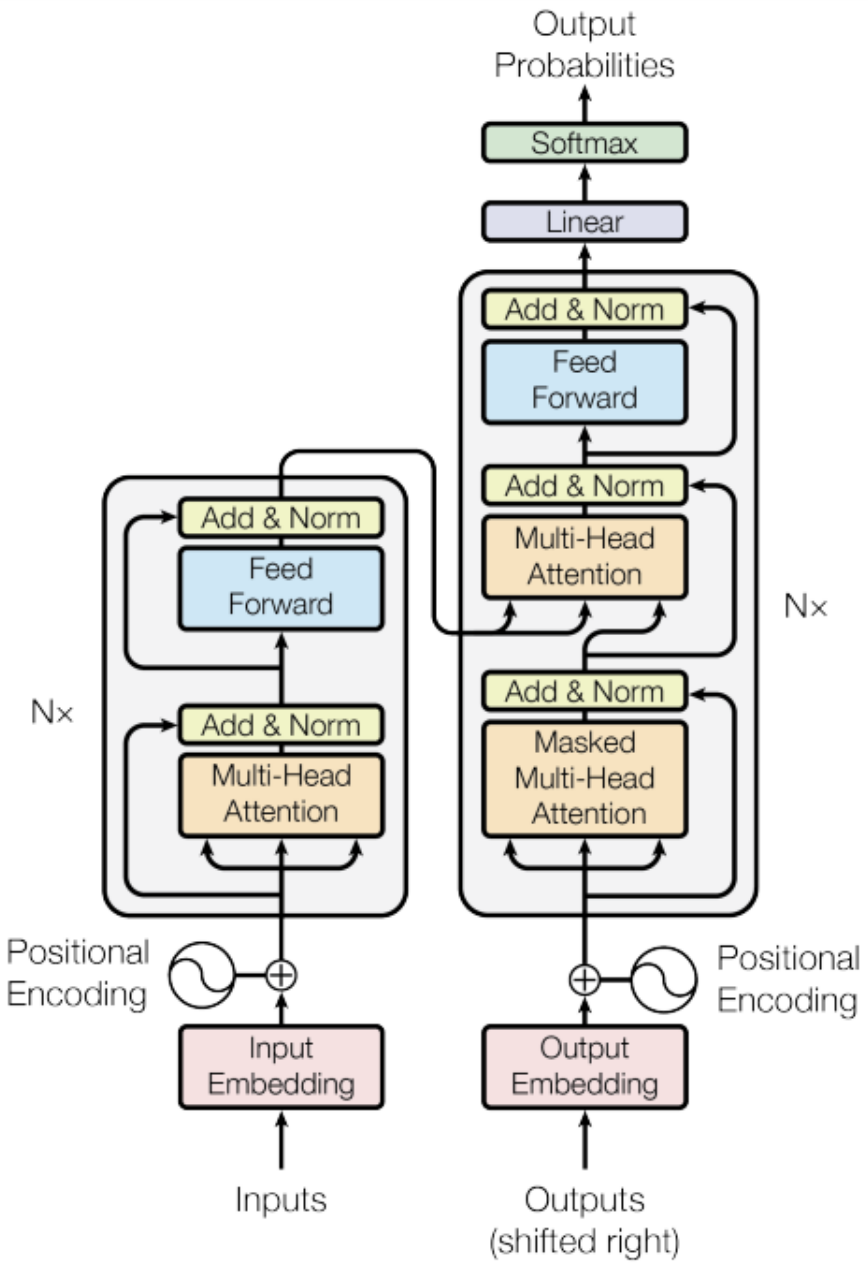
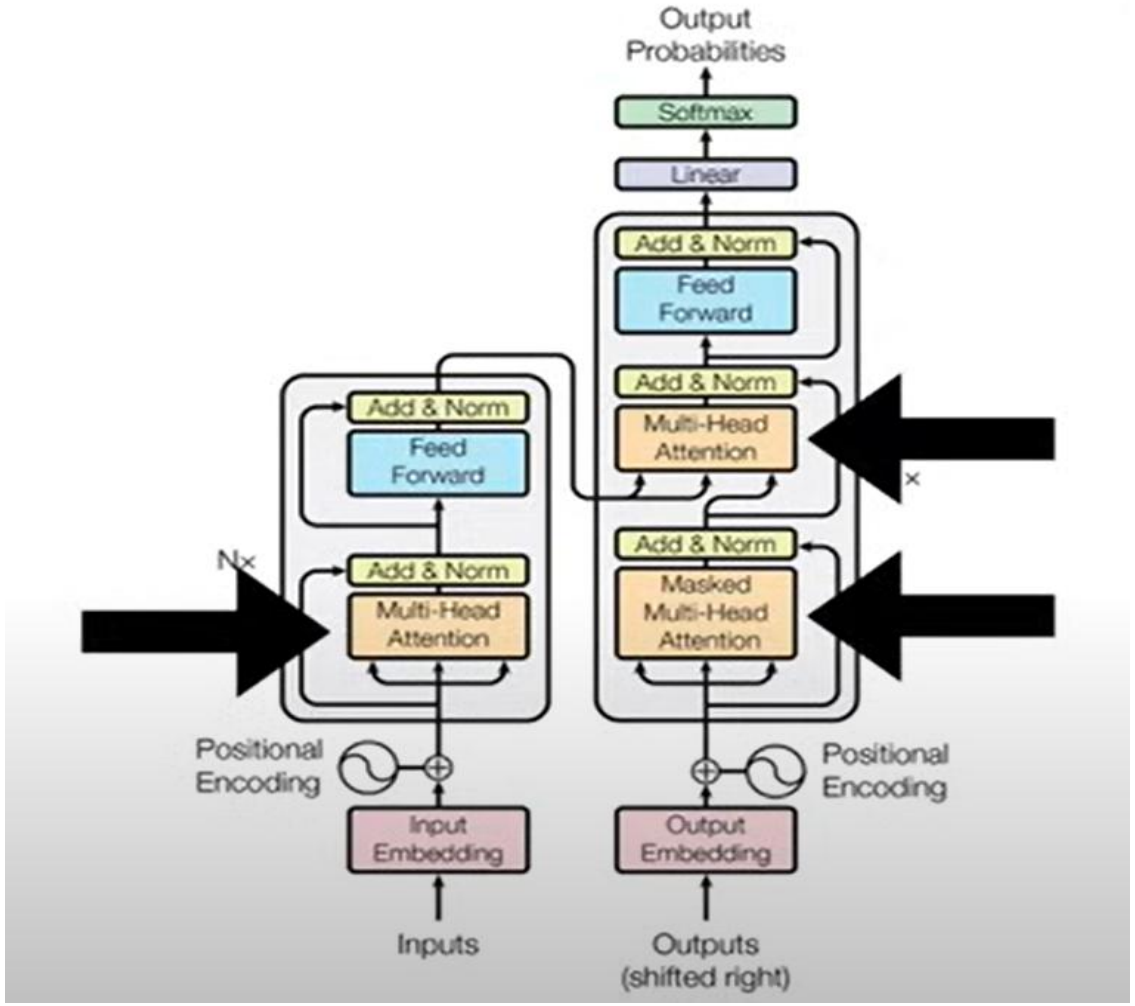


Figure 1: The Transformer - model architecture.

Transformers

Transformers apply a set of mathematical techniques called attention or self-attention to detect the subtle ways in which even distant data elements in a series influence and depend on each other.

Weight get trained with transformer model



Comparing RNNs to Transformer

RNNs

(+) LSTMs work reasonably well for long sequences.

(-) Expects an ordered sequences of inputs

(-) Sequential computation: subsequent hidden states can only be computed after the previous ones are done.

Transformer:

(+) Good at long sequences. Each attention calculation looks at all inputs.

(+) Can operate over unordered sets or ordered sequences with positional encodings.

(+) Parallel computation: All alignment and attention scores for all inputs can be done in parallel.

(-) Requires a lot of memory: N x M alignment and attention scalers need to be calculated and stored for a single self-attention head. (but GPUs are getting bigger and better)

Original paper

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

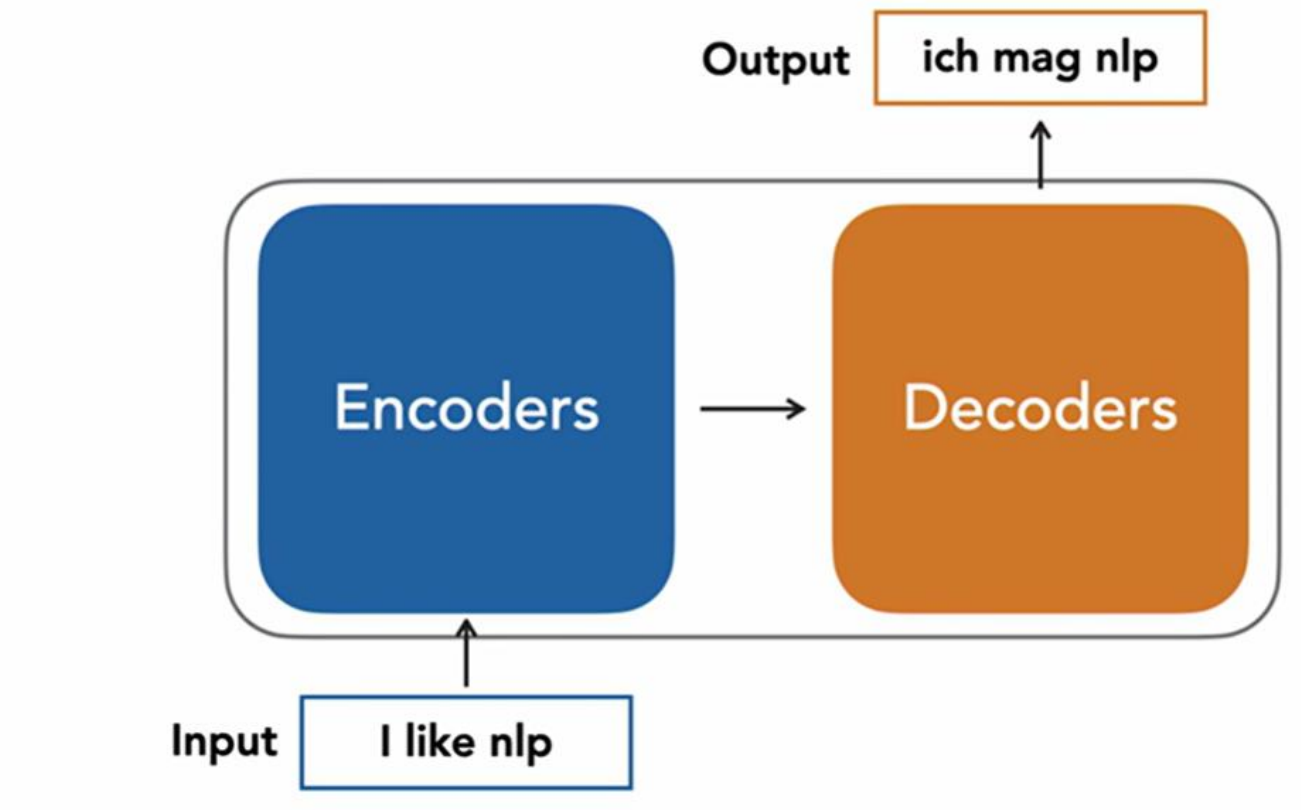
"ImageNet Moment for Natural Language Processing"

Pretraining:
Download a lot of text from the internet

Train a giant Transformer model for language modelin

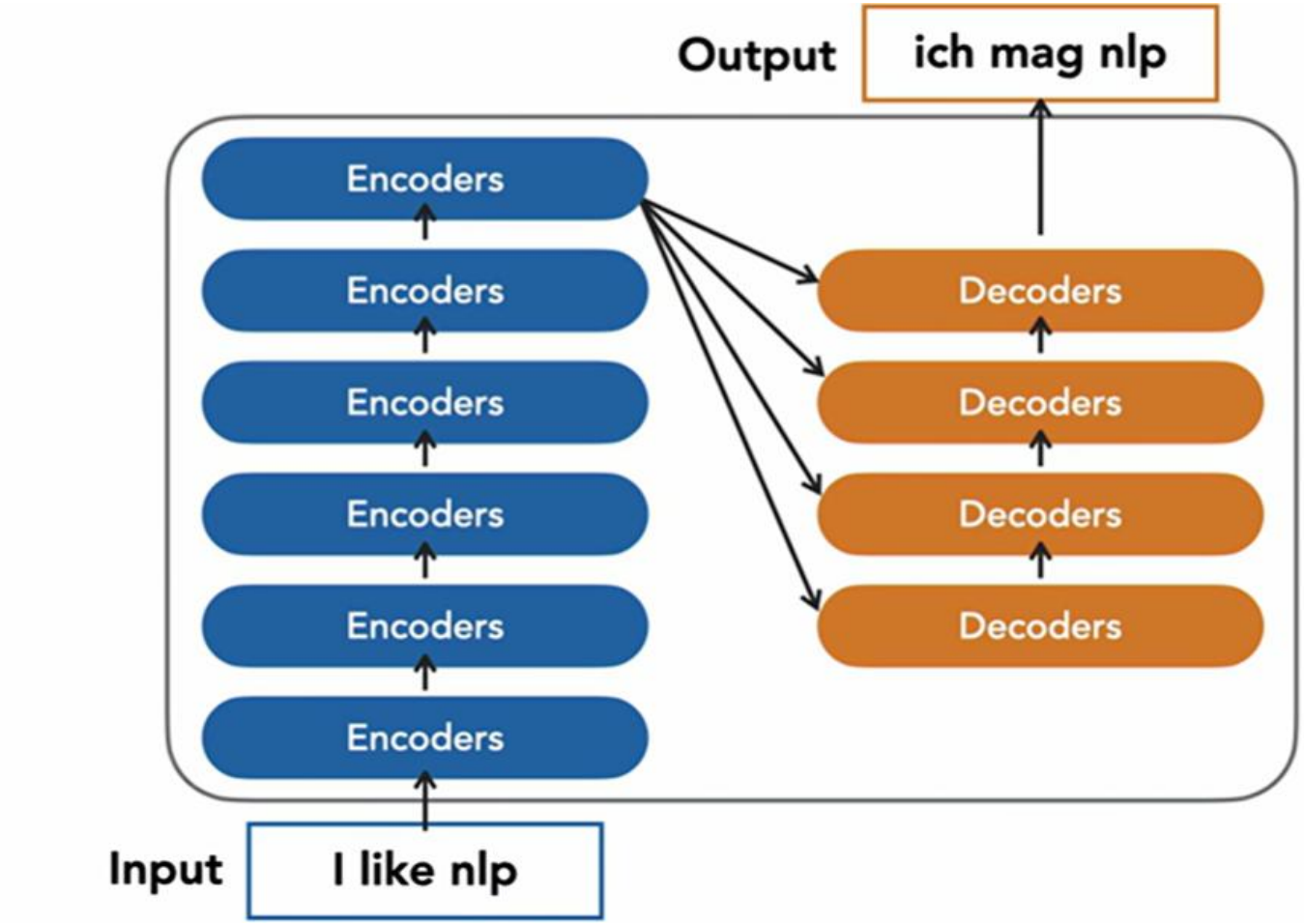
Finetuning:
Fine-tune the Transformer on your own NLP task

Transformer overview



Transformers are a type of layer that uses self-attention and layer norm.
It is highly scalable and highly parallelizable
Faster training, larger models, better performance across vision and language tasks

Transformer overview



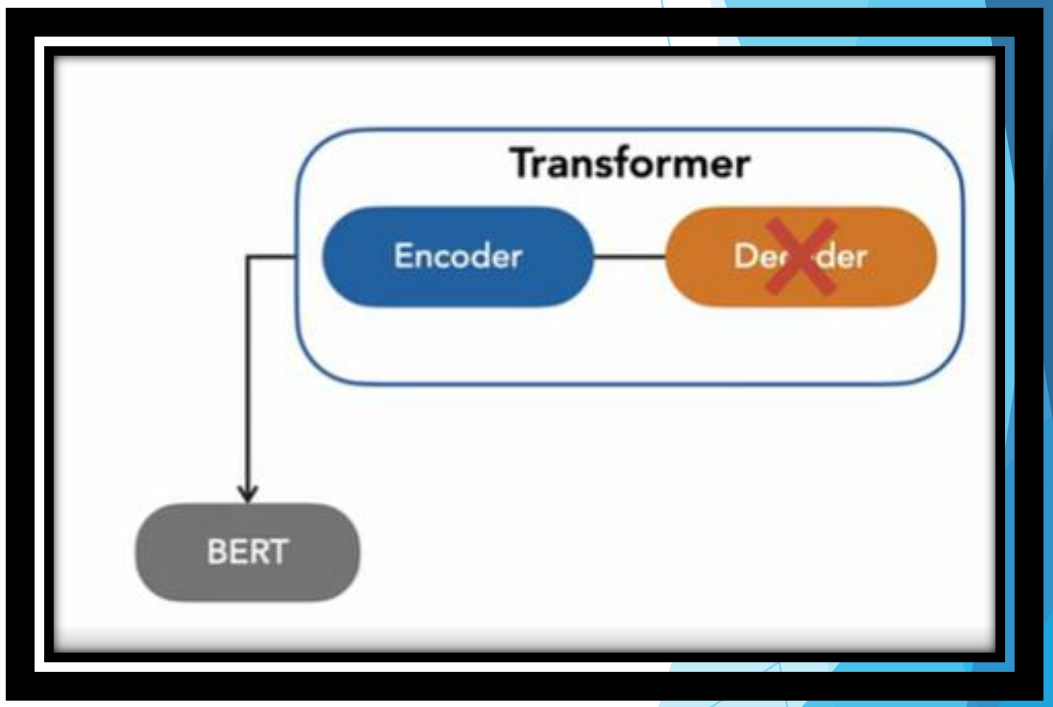
Popular Encoder decoder model

- Generative tasks
- BART (Bidirectional and Auto-Regressive Transformer)
- T5 T5 is an encoder-decoder model and converts all NLP problems into a text-to-text format.

Encoder only model

- Understanding of input
- Sentence classification
- Named entity recognition.

- Family of BERT model
- BERT
- RoBERTa
- DISTIL BIRT



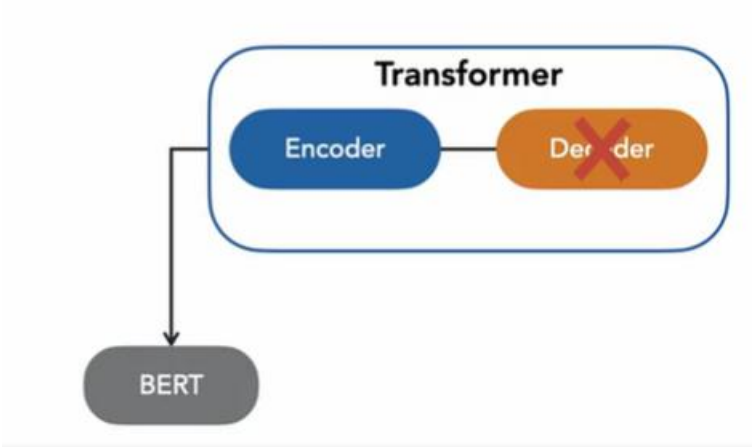
BERT does not do

- BERT does not do generate text.
- TEXT translation
- TEXT summarization.

BERT can do

Does:

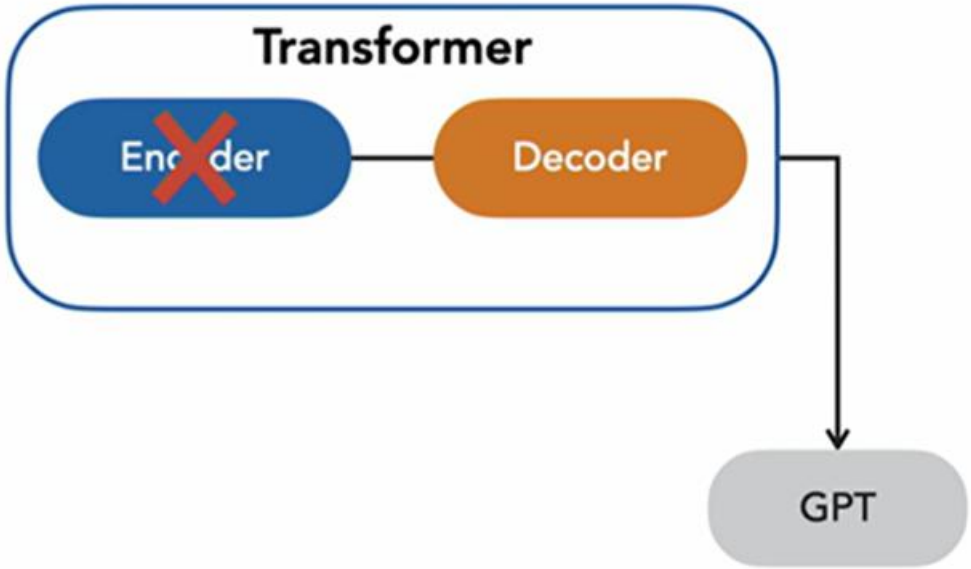
- Text classification
- Named entity recognition
- Question answering
- Fill in the blank



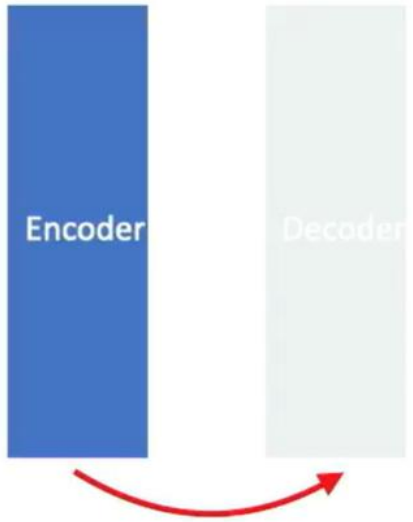
Decoder only model

- Good for generating task

- Such as
- GPT
- GPT2
- GPT3

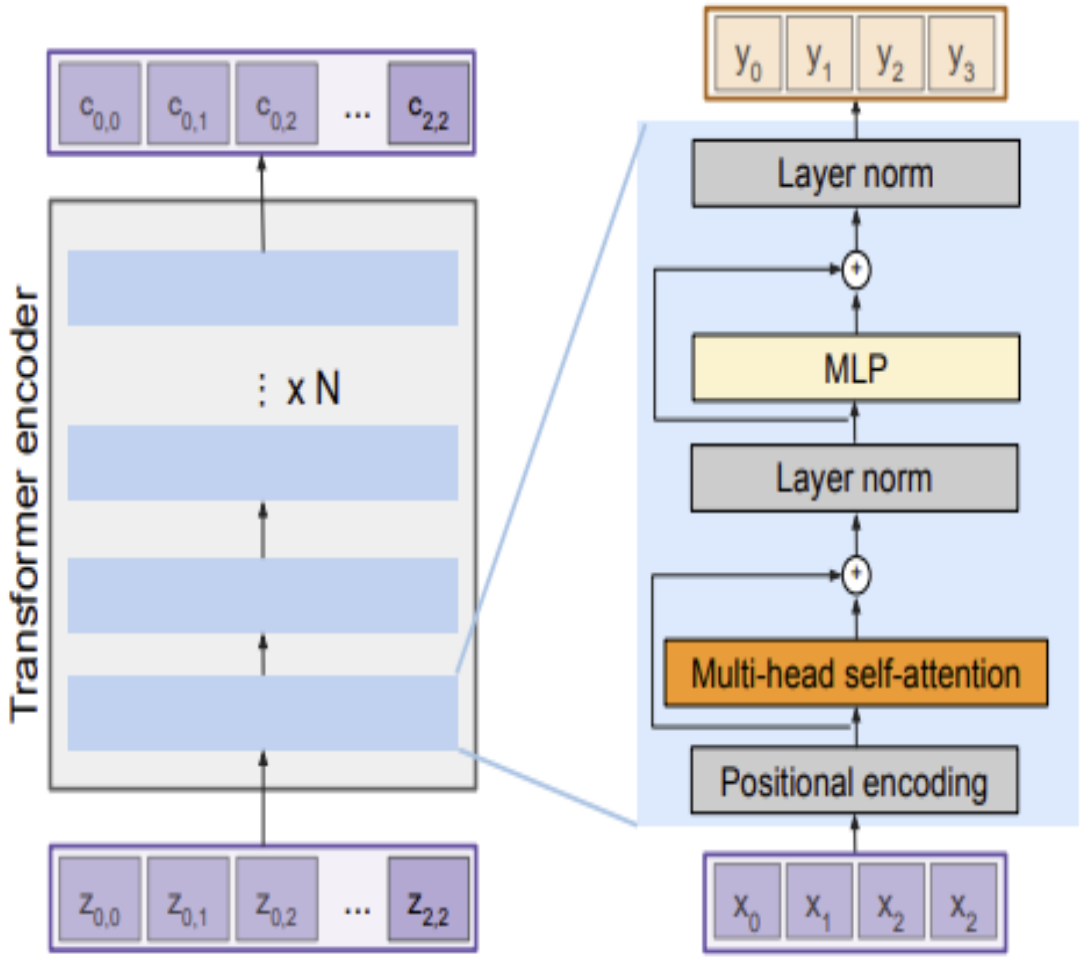


The Transformer encoder block



Receives an input and builds a representation of the input - model optimized to understand the input

The Transformer encoder block



Transformer Encoder Block:

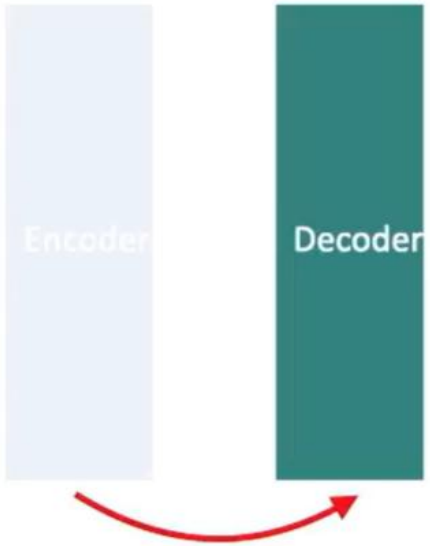
Inputs: Set of vectors **x**
Outputs: Set of vectors **y**

Self-attention is the only interaction between vectors.

Layer norm and MLP operate independently per vector.

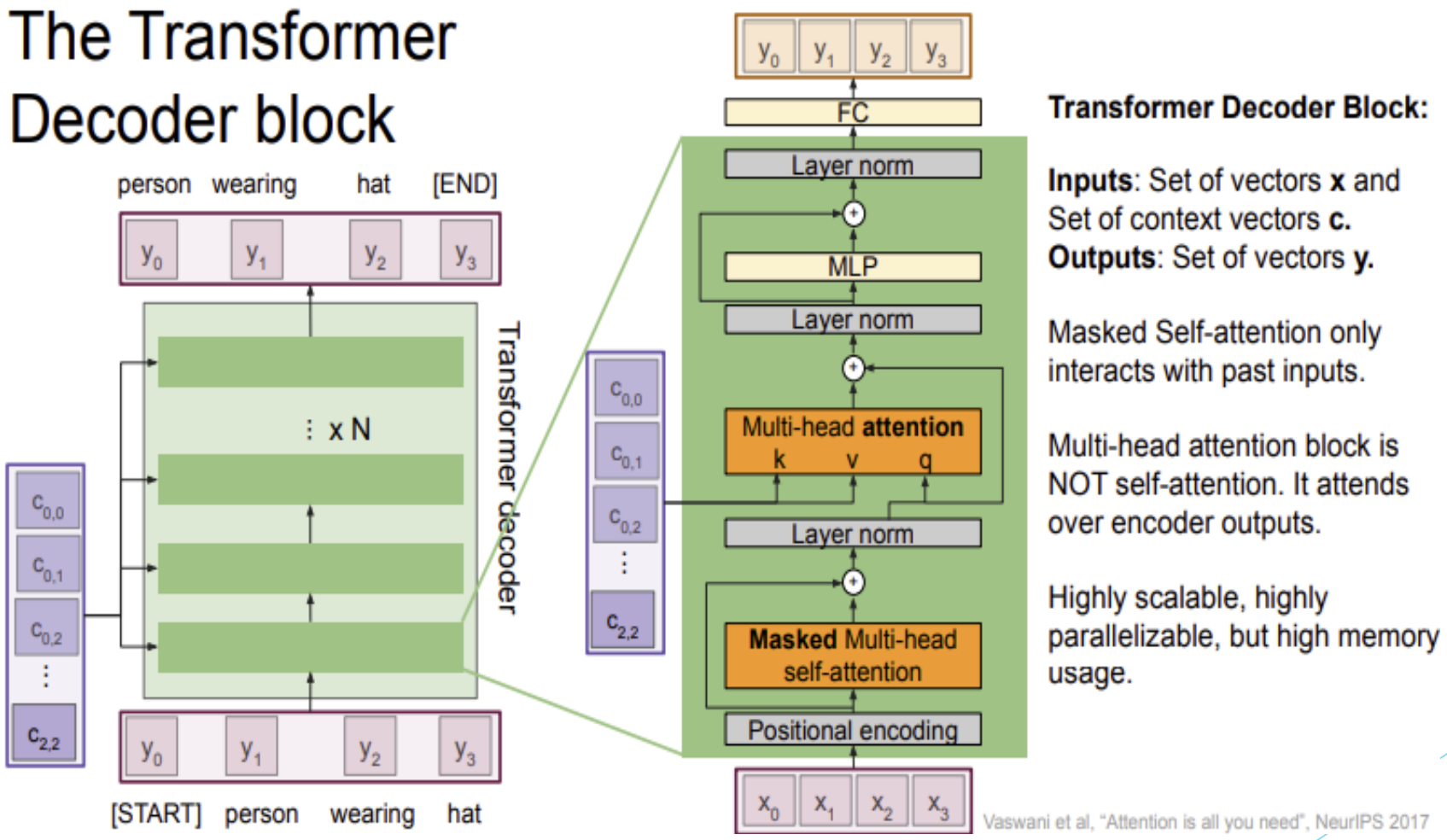
Highly scalable, highly parallelizable, but high memory usage.

Decoder



Decoder uses the encoder's representation to generate a target sequence - optimized for generating outputs

The Transformer Decoder block



Transformer Decoder Block:

Inputs: Set of vectors **x** and Set of context vectors **c.**
Outputs: Set of vectors **y.**

Masked Self-attention only interacts with past inputs.

Multi-head attention block is NOT self-attention. It attends over encoder outputs.

Highly scalable, highly parallelizable, but high memory usage.

Vaswani et al, "Attention is all you need", NeurIPS 2017

Pre-training

- pretraining—learning knowledge about language and the world from vast amounts of text—and call the resulting pretrained language models large language models.
- Large language models exhibit remarkable performance on all sorts of natural language tasks because of the knowledge they learn in pretraining,
- E.g: summarization, machine translation, question answering, or chatbots

The Transformer: A Self-Attention Network

- The standard architecture for building large language models is the transformer . The transformer makes use of a novel mechanism called self-attention.

- Transformers are a neural architecture that can handle distant information. But unlike LSTMs, transformers are not based on recurrent connections (which can be hard to parallelize), which means that transformers can be more efficient to implement at scale.

The Transformer: A Self-Attention Network

- Transformers are made up of stacks of transformer blocks, each of which is a multilayer network that maps sequences of input vectors (x1,...,xn) to sequences of output vectors (z1,...,zn) of the same length. These blocks are made by combining simple linear layers, feedforward networks, and self-attention layers, the key innovation of transformers. Self-attention allows a network to directly extract and use information from arbitrarily large contexts.

The Transformer: A Self-Attention Network

- Self-attention is just such a mechanism: it allows us to look broadly in the context and tells us how to integrate the representation from words in that context from layer k −1 to build the representation for words in layer k.

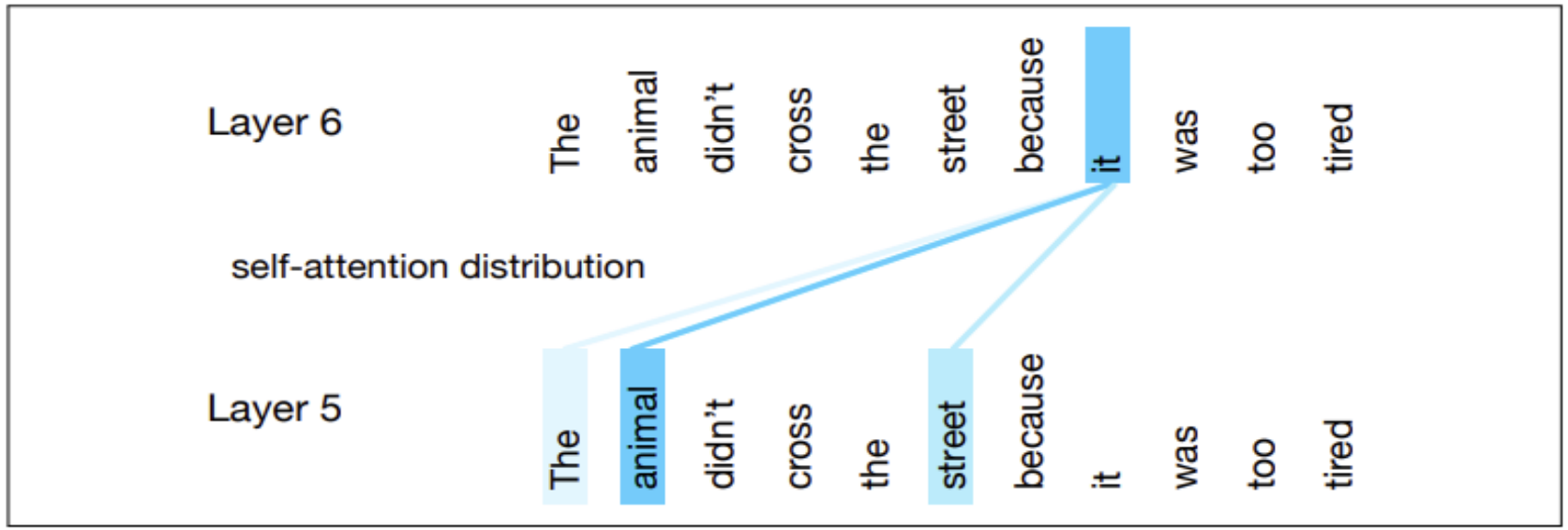


Figure 10.1 The self-attention weight distribution α that is part of the computation of the representation for the word *it* at layer 6. In computing the representation for *it*, we attend differently to the various words at layer 5, with darker shades indicating higher self-attention values. Note that the transformer is attending highly to *animal*, a sensible result, since in this example *it* corefers with the animal, and so we'd like the representation for *it* to draw on the representation for *animal*. Figure simplified from (Uszkoreit, 2017).

The Transformer: A Self-Attention Network

- Here we want to compute a contextual representation for the word it, at layer 6 of the transformer, and we'd like that representation to draw on the representations of all the prior words, from layer 5. The figure uses color to represent the attention distribution over the contextual words: the word animal has a high attention weight, meaning that as we are computing the representation for it, we will draw most heavily on the representation for animal. This will be useful for the model to build a representation that has the correct meaning for it, which indeed is co-referent here with the word animal.

Causal or backward-looking self-attention

- In causal, or backward looking self-attention, the context is any of the prior words. In general bidirectional self-attention, the context can include future words.

Fig. 10.2 thus illustrates the flow of information in a single causal, or backward looking, self-attention layer. As with the overall transformer, a self-attention layer maps input sequences (x1,...,xn) to output sequences of the same length (a1,...,an). When processing each item in the input, the model has access to all of the inputs up to and including the one under consideration, but no access to information about inputs beyond the current one. In addition, the computation performed for each item is independent of all the other computations. The first point ensures that we can use this approach to create language models and use them for autoregressive generation, and the second point means that we can easily parallelize both forward inference and training of such models.

Causal or backward-looking self-attention

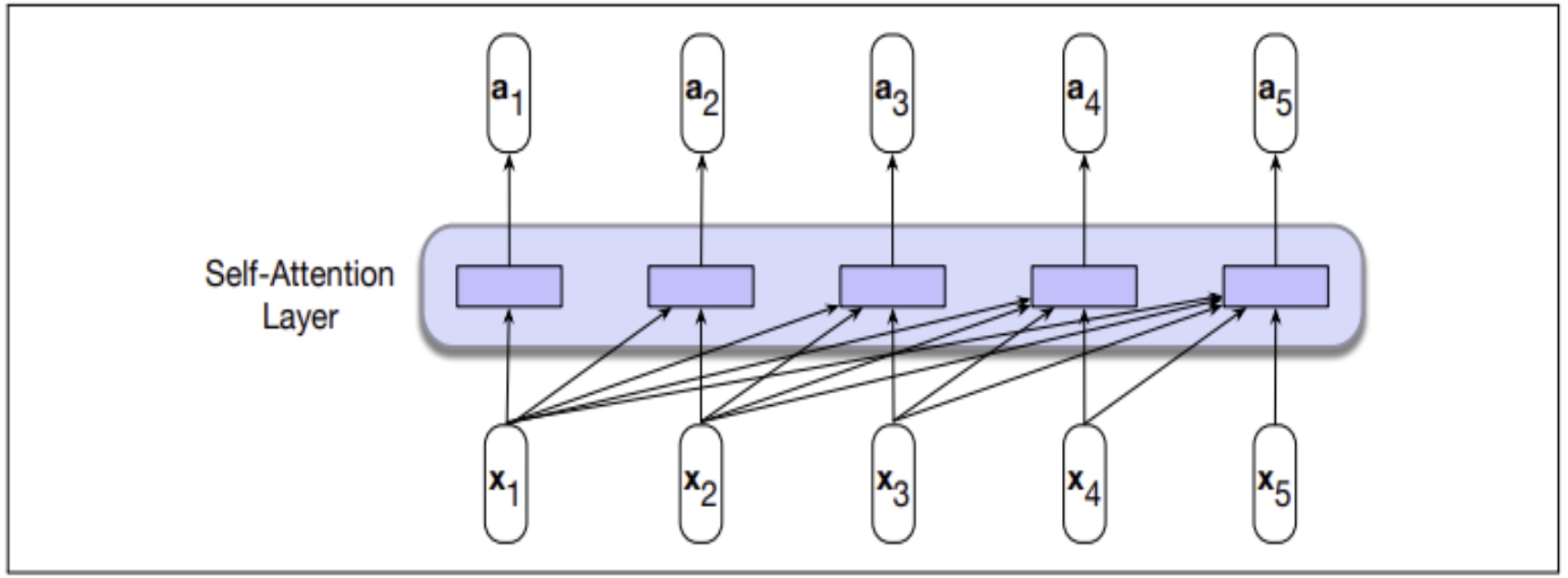


Figure 10.2 Information flow in a causal (or masked) self-attention model. In processing each element of the sequence, the model attends to all the inputs up to, and including, the current one. Unlike RNNs, the computations at each time step are independent of all the other steps and therefore can be performed in parallel.

Attention based approach review

- How shall we compare words to other words? Since our representations for words are vectors, we'll make use of our old friend the dot product that we used for computing word similarity

$$\textit{Verson 1:} \quad \text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j \tag{10.4}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \ \forall j \leq i \tag{10.5}$$

$$= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_{k=1}^{i} \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \ \forall j \leq i \tag{10.6}$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j \tag{10.7}$$

The steps embodied in Equations 10.4 through 10.7 represent the core of an attention-based approach: a set of comparisons to relevant items in some context, a normalization of those scores to provide a probability distribution, followed by a weighted sum using this distribution. The output a is the result of this straightforward computation over the inputs

Transformer

- As the current focus of attention when being compared to all of the other query preceding inputs. We'll refer to this role as a query.
- In its role as a preceding input being compared to the current focus of attention. We'll refer to this role as a key.
- And finally, as a value used to compute the output for the current focus of attention

To capture these three different roles, transformers introduce weight matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V$. These weights will be used to project each input vector $\mathbf{x}_i$ into a representation of its role as a key, query, or value.

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W}^Q; \quad \mathbf{k}_i = \mathbf{x}_i\mathbf{W}^K; \quad \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^V \tag{10.8}$$

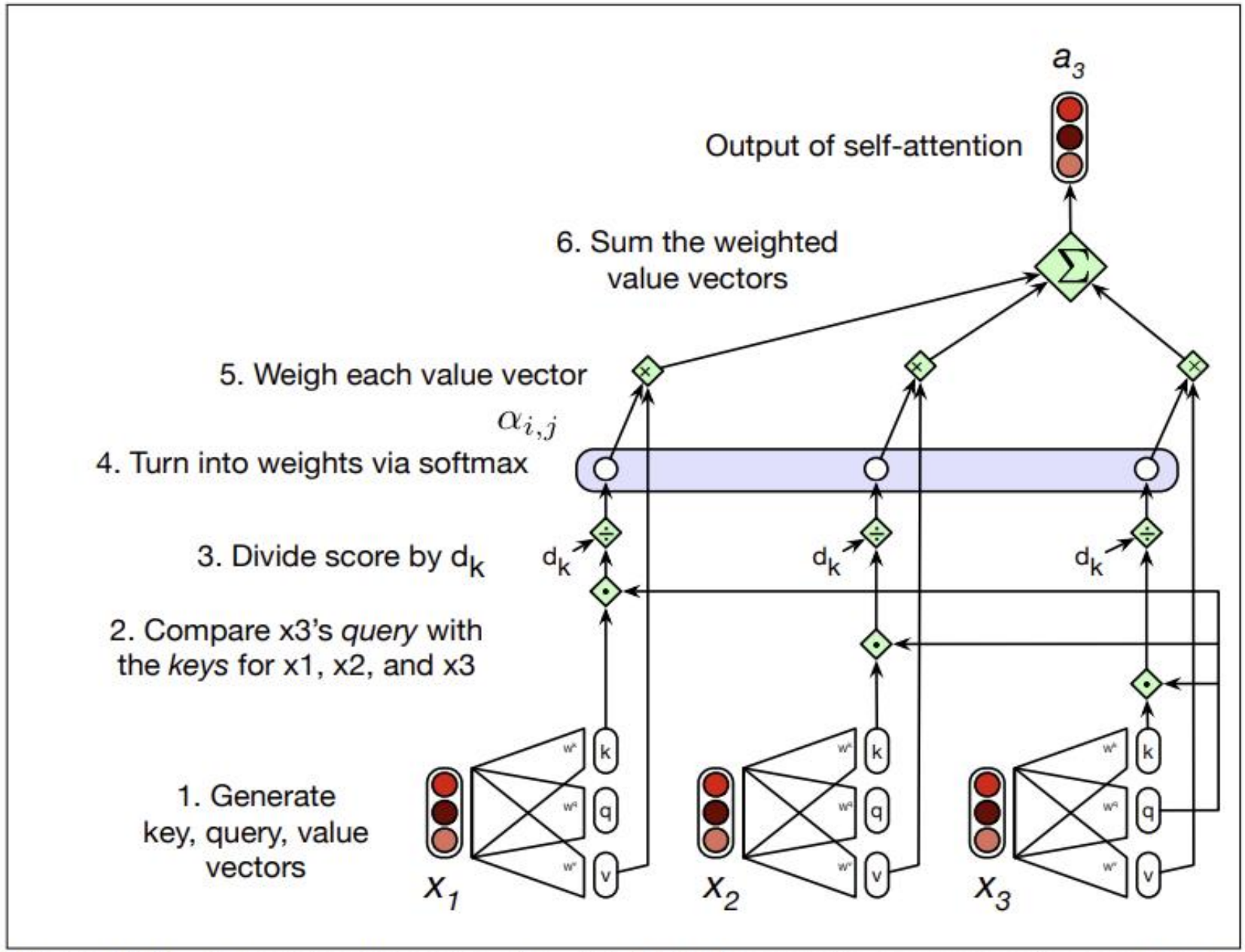


Figure 10.3 Calculating the value of $\mathbf{a}_3$, the third element of a sequence using causal (left-to-right) self-attention.

Transformer

- There is one final part of the self-attention model. The result of a dot product can be an arbitrarily large (positive or negative) value. Exponentiating large values can lead to numerical issues and to an effective loss of gradients during training. To avoid this, we scale down the result of the dot product, by dividing it by a factor related to the size of the embeddings.

A typical approach is to divide by the square root of the dimensionality of the query and key vectors (dk), leading us to update our scoring function one more time, replacing Eq. 10.4 and Eq. 10.9 with Eq. 10.12.

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W}^{\mathbf{Q}}; \mathbf{k}_i = \mathbf{x}_i\mathbf{W}^{\mathbf{K}}; \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^{\mathbf{V}} \quad (10.11)$$

$$\textit{Final verson:} \quad \text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \quad (10.12)$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \ \forall j \leq i \quad (10.13)$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij}\mathbf{v}_j \quad (10.14)$$

Parallelizing self-attention using a single matrix X

- This description of the self-attention process has been from the perspective of computing a single output at a single time step i. However, since each output, y_i , is computed independently, this entire process can be parallelized, taking advantage of efficient matrix multiplication routines by packing the input embeddings of the N tokens of the input sequence into a single matrix X ∈ R N×d . That is, each row of X is the embedding of one token of the input. Transformers for large language models can have an input length N = 1024, 2048, or 4096 tokens, so X has between 1K and 4K rows, each of the dimensionality of the embedding d. Given these matrices we can compute all the requisite query-key comparisons simultaneously by multiplying Q and K| in a single matrix multiplication (the product is of shape N × N

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{\mathbf{Q}};\ \mathbf{K} = \mathbf{X}\mathbf{W}^{\mathbf{K}};\ \mathbf{V} = \mathbf{X}\mathbf{W}^{\mathbf{V}} \tag{10.15}$$

$$\mathbf{A} = \text{SelfAttention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^{\mathsf{T}}}{\sqrt{d_k}}\right)\mathbf{V} \tag{10.16}$$

Multi-head Attention

- Transformers actually compute a more complex kind of attention than the single self-attention calculation we've seen so far. This is because the different words in a sentence can relate to each other in many different ways simultaneously.

- It would be difficult for a single self-attention model to learn to capture all of the different kinds of parallel relations among its inputs. Transformers address this issue with multi-head self-attention layers. These multi-head self-attention layers are sets of self-attention layers, called heads, that reside in parallel layers at the same depth in a model, each with its own set of parameters

Multi-head Attention

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_i^Q ; \mathbf{K} = \mathbf{X}\mathbf{W}_i^K ; \mathbf{V} = \mathbf{X}\mathbf{W}_i^V \quad (10.17)$$

$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \quad (10.18)$$

$$\mathbf{A} = \text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \ldots \oplus \mathbf{head}_h)\mathbf{W}^O \quad (10.19)$$

Fig. 10.5 illustrates this approach with 4 self-attention heads. In general in transformers, the multihead layer is used instead of a self-attention layer.

Transformer Blocks

- The self-attention calculation lies at the core of what's called a transformer block, which, in addition to the self-attention layer, includes three other kinds of layers: (1) a feedforward layer, (2) residual connections, and (3) normalizing layers (colloquially called "layer norm").

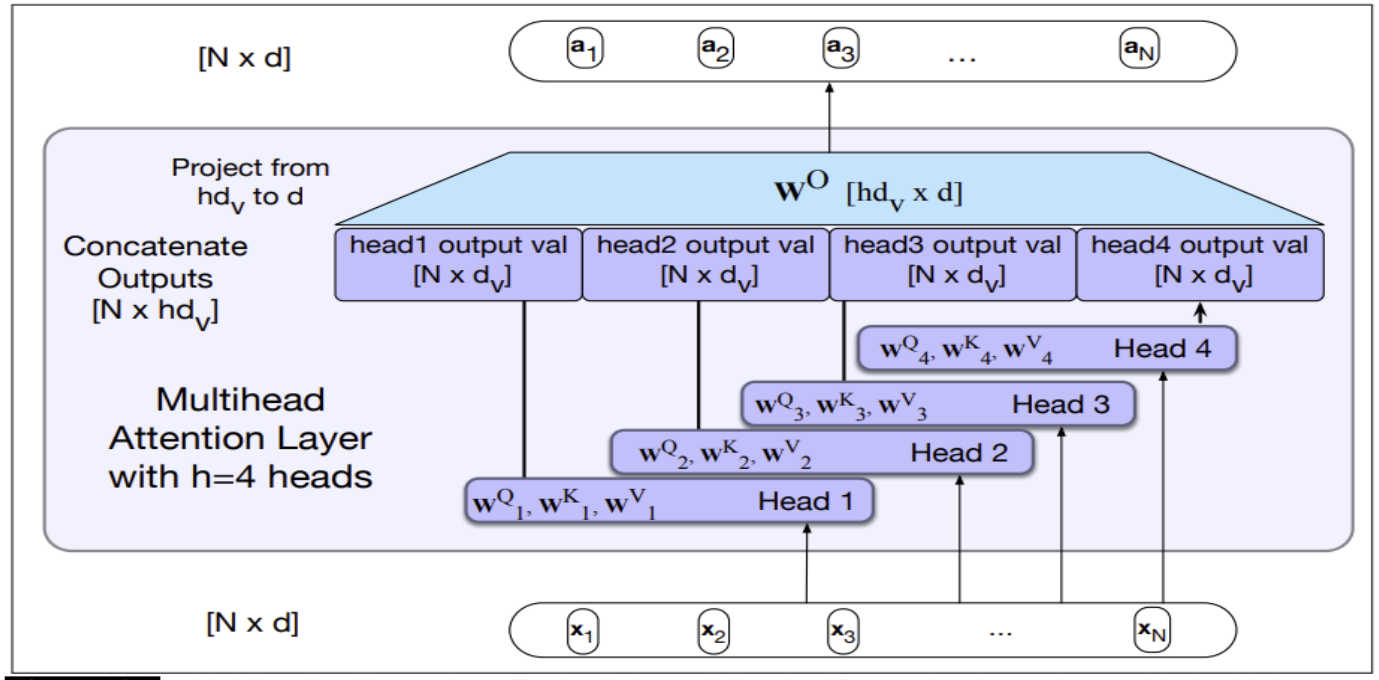


Figure 10.5 Multihead self-attention: Each of the multihead self-attention layers is provided with its own set of key, query and value weight matrices. The outputs from each of the layers are concatenated and then projected to d, thus producing an output of the same size as the input so the attention can be followed by layer norm and feedforward and layers can be stacked.

Transformer Blocks

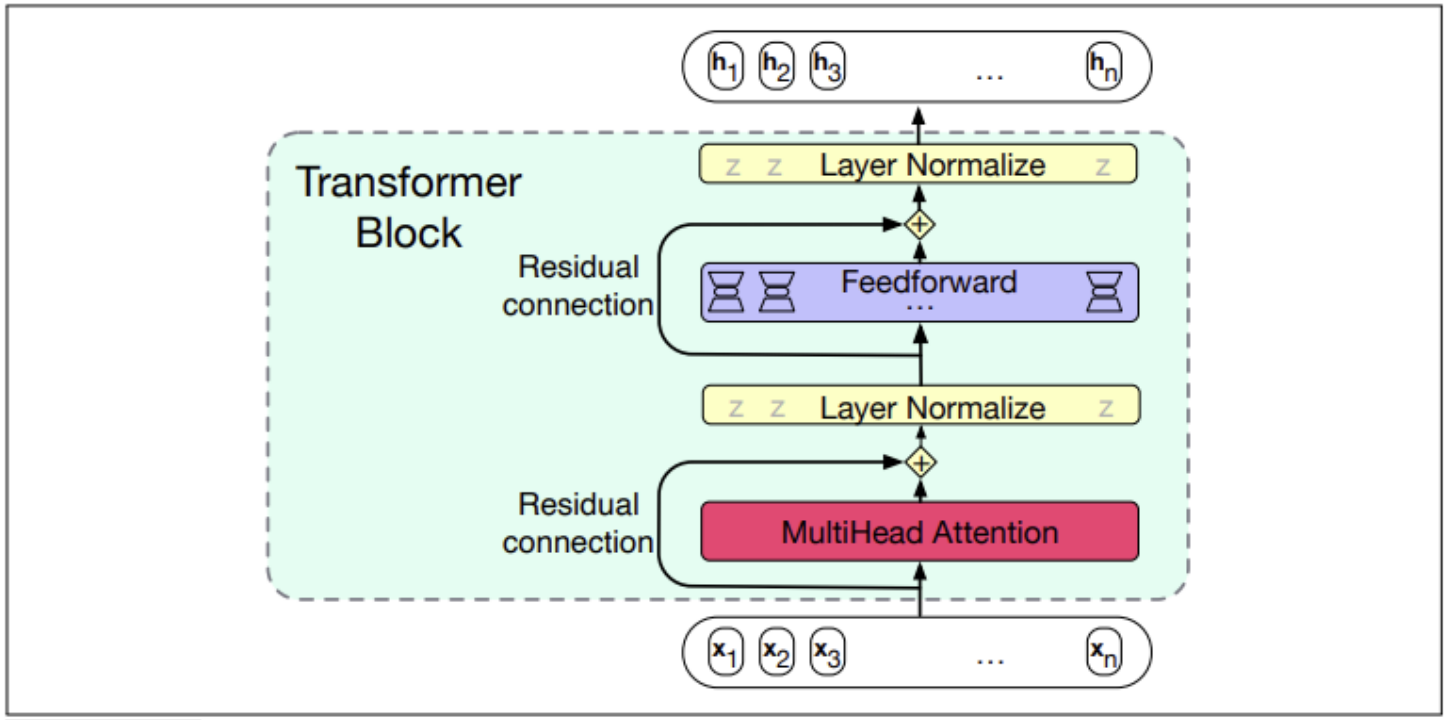


Figure 10.6 A transformer block showing all the layers.

Transformer Blocks

Putting it all together The function computed by a transformer block can be expressed as:

$$\mathbf{O} = \text{LayerNorm}(\mathbf{X} + \text{SelfAttention}(\mathbf{X})) \quad (10.24)$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{O} + \text{FFN}(\mathbf{O})) \quad (10.25)$$

Or we can break it down with one equation for each component computation, using $\mathbf{T}$ (of shape $[N \times d]$) to stand for transformer and superscripts to demarcate each computation inside the block:

$$\mathbf{T}^1 = \text{SelfAttention}(\mathbf{X}) \quad (10.26)$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1 \quad (10.27)$$

$$\mathbf{T}_3 = \text{LayerNorm}(\mathbf{T}^2) \quad (10.28)$$

$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3) \quad (10.29)$$

$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3 \quad (10.30)$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5) \quad (10.31)$$

Transformer Blocks

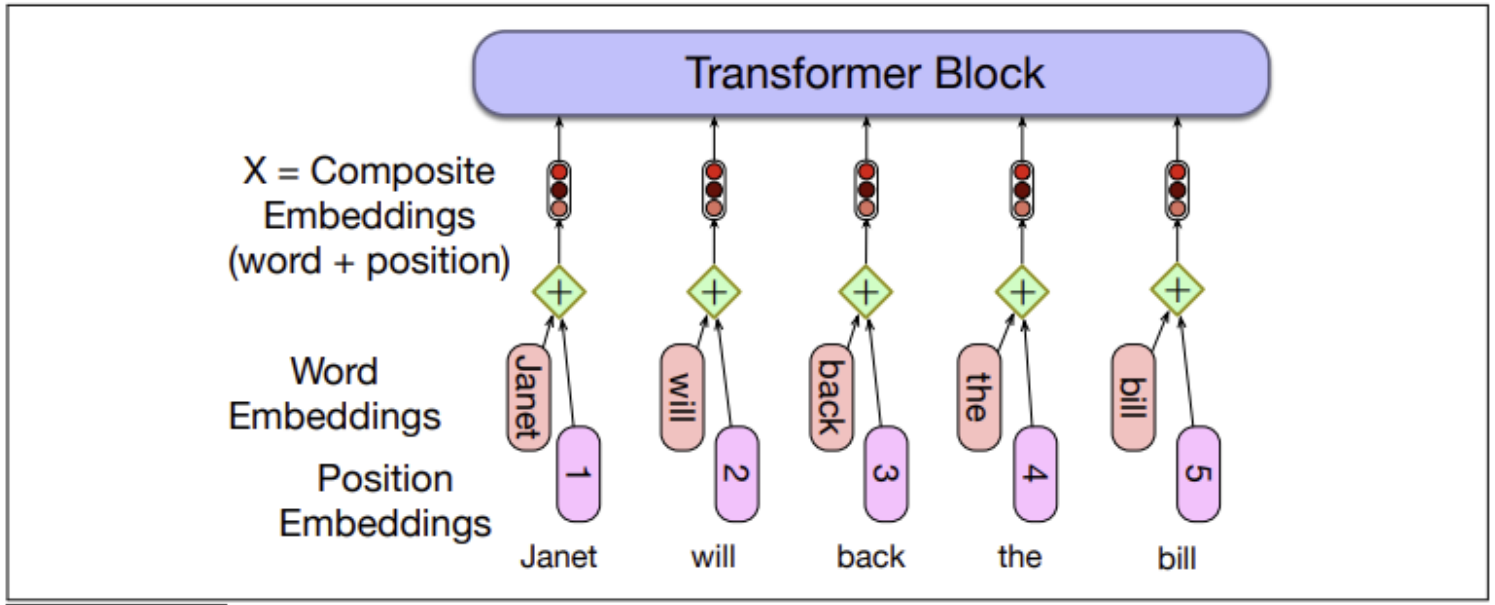


Figure 10.12 A simple way to model position: add an embedding of the absolute position to the token embedding to produce a new embedding of the same dimenionality.

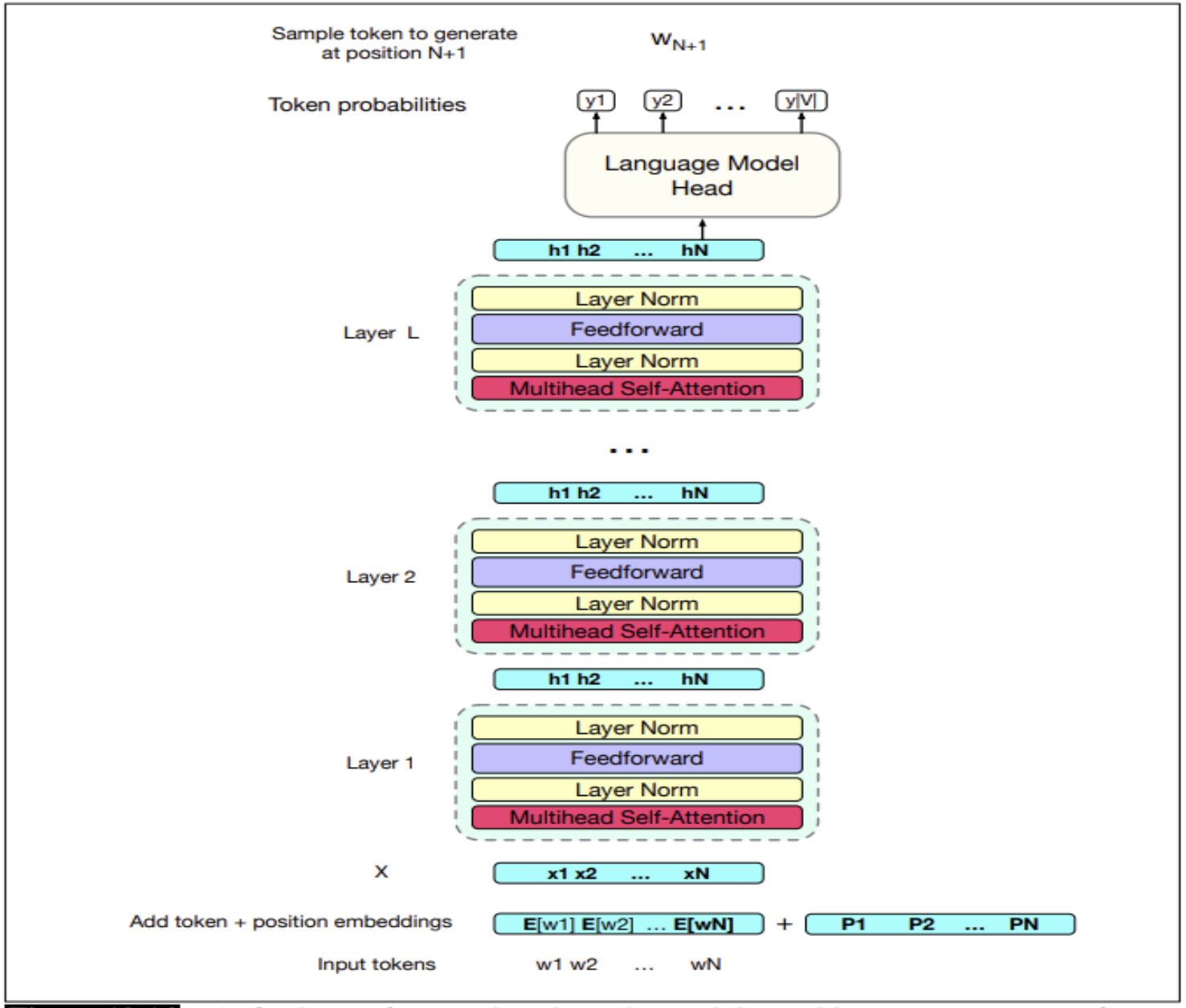
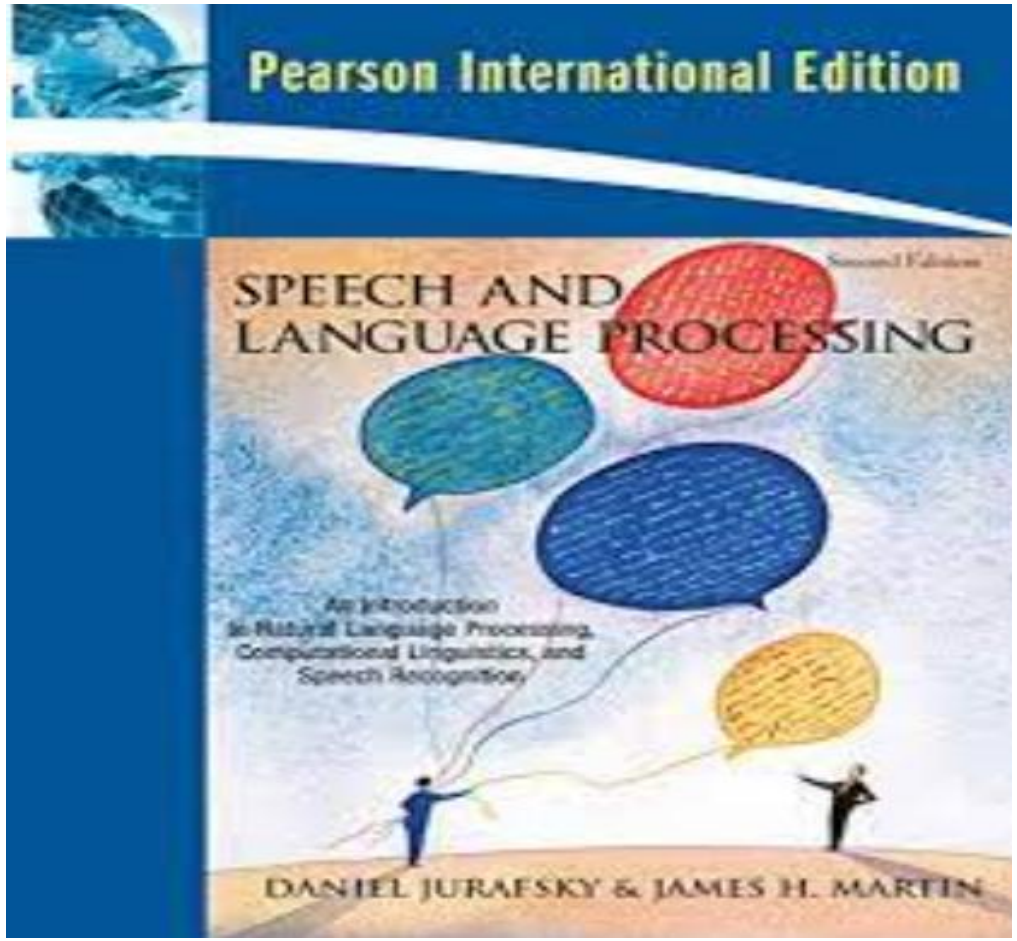


Figure 10.14 A final transformer decoder-only model, stacking post-norm transformer blocks and mapping from a set of input tokens w_1 to w_N to a predicted next word w_{N+1}.

Reference



Chapter 10

Question

Thank you